Module - 1

Introduction

Contents

- Definition of an operating system
- Abstract views of an operating system
- Functions of an operating system
- Event-driven systems
- Efficiency & system performance goals
- Evolution of operating system designs
- Classes of operating systems and examples of operating systems.

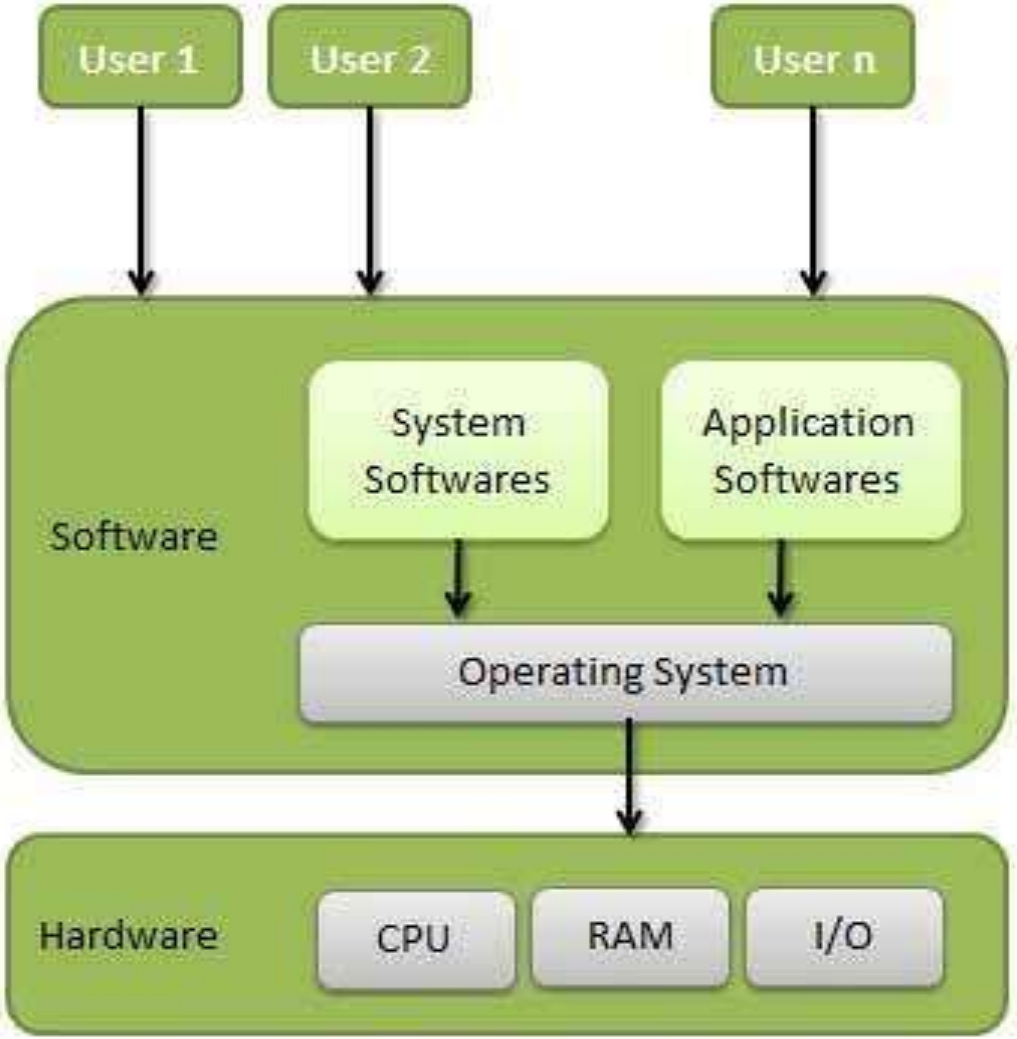
What is an Operating system?

- A program that acts as an intermediary between a user of a computer and the computer hardware
- Operating system goals:
  - **Execute user programs** and make solving user problems easier
  - Make the computer **system convenient to use**
  - Use the computer **hardware in an efficient manner**
- An operating system is software which performs all the basic tasks like file management, memory management, process management, handling input and output, and controlling peripheral devices such as disk drives and printers.
- Some popular Operating Systems include Linux, Windows, OS X, VMS, OS/400, AIX, z/OS, etc.

Computer System Structure

- Computer system can be divided into four components:
  - Hardware – provides basic computing resources
    - CPU, memory, I/O devices
  - Operating system
    - Controls and coordinates use of hardware among various applications and users
  - Application programs – define the ways in which the system resources are used to solve the computing problems of the users
    - Word processors, compilers, web browsers, database systems, video games
  - Users
    - People, machines, other computers

Four Components of a Computer System



What Operating Systems Do

- Operating system provides the means for proper use of resources in the operation of computer system.
- Its like government it provides an environment within which user can execute programs.
- The role of operating system can be in two views
    1. User View
    2. System View

What Operating Systems Do cont…

User view:

1. **Single User View Point**

- For **PC's OS is designed for convenience** ease of use.
- Some attention on performance rather than resource utilization.
- Systems are optimized for single user rather than multiple users.

2. **Multiple User View Point**

- Designed for client server architectures.
- Importance given to resource allocation.
- Effective use of resources with good performance by multiple users.

3. Handled User View Point

- Designed for handheld devices like mobile phones and computers.
- Touch screens as user interface.
- Connected with other devices via wireless to perform various operations.
- Not efficient compared to computer interface.

4. Embedded System User View Point

- Lack of user interface.
- The remote control used to turn **on** or **off** the tv is all part of an embedded system in which the electronic device communicates with another program.

What Operating Systems Do cont..

System view

- OS is a **resource allocator**
  - Manages all resources
  - Decides between conflicting requests for efficient and fair resource use.

OS is a **control program**

- Controls execution of programs to prevent errors and improper use of the computer

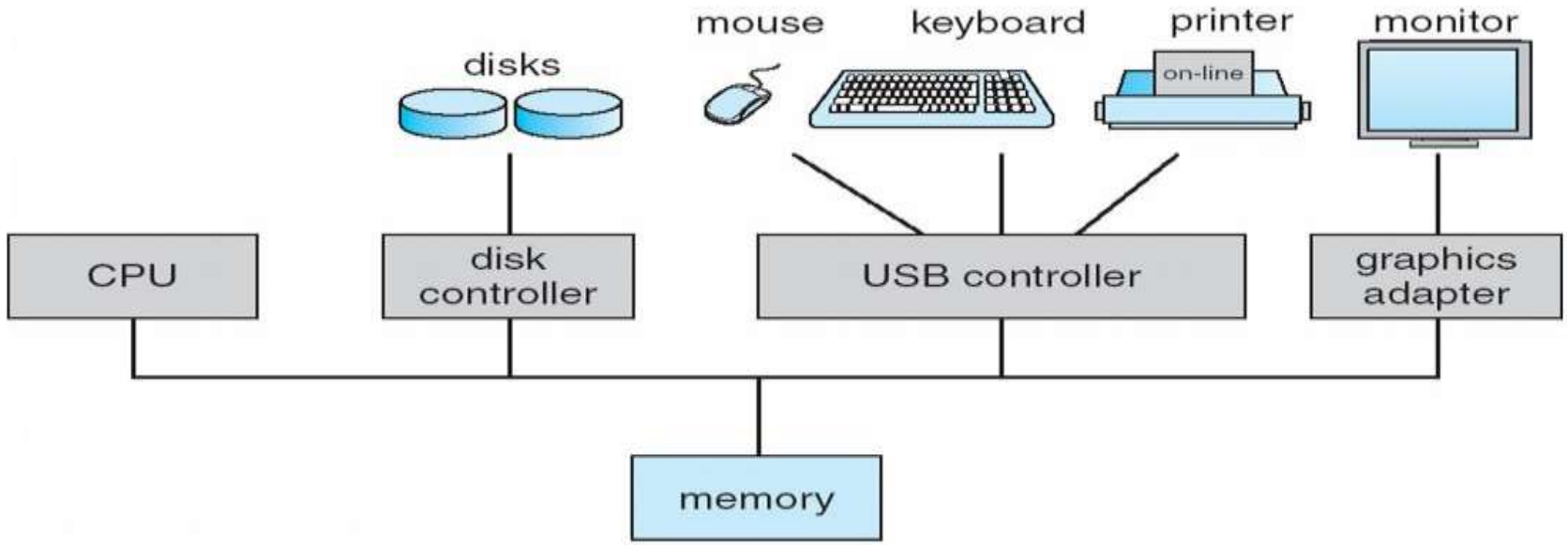
COMPUTER-SYSTEM ORGANIZATION

Computer system organization has several parts:

1. Computer system operation
2. Storage structure
3. I/O structure

Computer system operation

- One or more CPUs, **device driver controllers** connect through common bus providing access to shared memory .
- CPU and device controllers can execute in parallel, competing for memory cycles. To ensure orderly access to shared memory, a memory controller synchronizes access to the memory.
- Each device controller has a local buffer
- CPU moves data from/to main memory to/from local buffers



- Each device controller is in charge of a specific type of device (for example, a disk drive, audio device, or graphics display).

- Depending on the controller, more than one device may be attached. For instance, one system USB port can connect to a USB hub, to which several devices can connect.

- A device controller maintains some local buffer storage and a set of special-purpose registers.

- The device controller is responsible for moving the data between the peripheral devices that it controls and its local buffer storage.

- Operating systems have a device driver for each device controller.
- This device driver understands the device controller and provides the rest of the operating system with a uniform interface to the device.
- The CPU and the device controllers can execute in parallel, competing for memory cycles.
- To ensure orderly access to the shared memory, a memory controller synchronizes access to the memory.

- A boot strap program which stats up the computer stored in ROM or EPROM or EEPROM generally known as firmware.
- It loads operating system kernel into main memory and starts system execution. This process is called booting.
- Once kernel is loaded, OS starts providing services to the system and users.
- Some services are provided system programs outside the kernel those are called system processes or system daemons.
- Device controller informs CPU that it has finished its operation by causing An **interrupt**
- Hardware may trigger interrupt at any time by sending signal to CPU through system bus.
- Software may trigger interrupt by executing a special operation called system call or monitor call.

Some important terms:

1) Bootstrap Program:→The initial program that runs when a computer is powered up or rebooted.
→It is stored in the ROM.
→It must know how to load the OS and start executing that system.
→It must locate and load into memory the OS Kernel.

2) Interrupt: → The occurence of an event is usually signalled by an Interrupt from Hardware or Software.
→ Hardware may trigger an interrupt at any time by sending a signal to the CPU, usually by the way of the system bus.

3) System Call (Monitor call): → Software may trigger an interrupt by executing a special operation called System Call.

When the CPU is interrupted, it stops what it is doing and immediately transfers execution to a fixed location.

→ The fixed location usually contains the starting address where the Service Routine of the interrupt is located.

The Interrupt Service Routine executes.

On completion, the CPU resumes the interrupted computation.

Interrupts

1. Application Makes I/O Request

- The application program wants to read from a keyboard (or write to a disk), so it makes a request.

2. Device Driver Loads Registers

- The **device driver** (part of the OS) sends commands to the **device controller** by writing values into its registers (like “start reading”).

3. Device Controller Starts Operation

- The **device controller** performs the actual operation — for example, reading a character from the keyboard into its **local buffer**.

4. Interrupt Notifies Completion

- Once the controller finishes the job, it **raises an interrupt** to notify the **CPU**.

5. Interrupt Handler in OS Responds

- The **CPU pauses current execution** and invokes the **Interrupt Service Routine (ISR)**, which tells the **device driver** that the operation is complete.

6. Driver Returns Result

- The driver might return:
- The **data** (like a key pressed), or
- A **status** message (e.g., "Print completed").

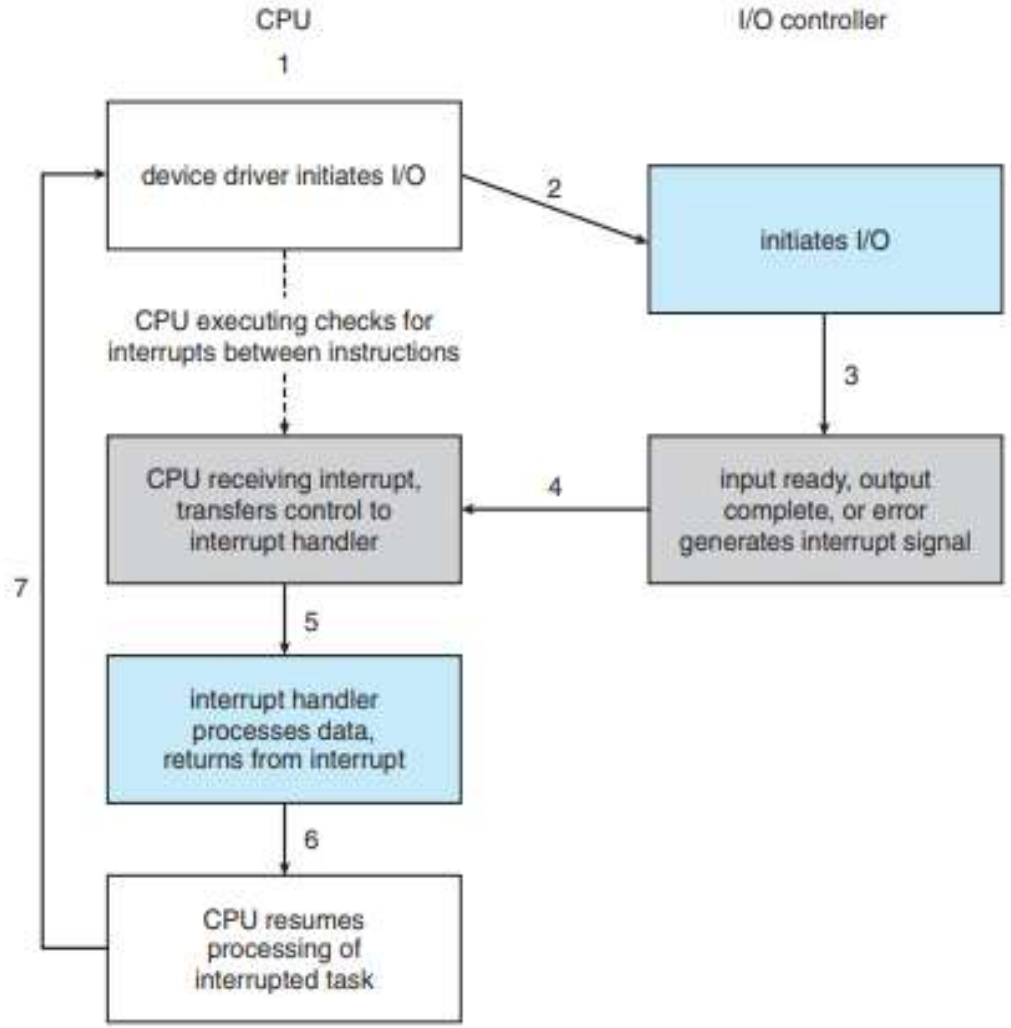


Figure 1.4 Interrupt-driven I/O cycle.

I/O Structure

- Computer system consists of CPU and multiple device controllers connected through common bus.
- More devices are attached to small computer system interface(SCSI) controller.
- Device controller maintains local buffer storage and set of special purpose registers.
- DC is responsible for moving data b/w peripherals and its local buffer.
- Operating system have a device driver for each device controller.
- To start I/O operation the device driver loads the appropriate registers within device controller.
- The DC examine the contents of these registers and determine what action to take.

- The controller starts the transfer of data from the device to buffer. Once data transfer complete, the device controller informs the device driver via an interrupt that it has finished its operation.
- The device driver then returns the control to OS, by returning data or pointer to data.
- This form of interrupt driven I/O is suitable for moving small amounts of data but failed to move bulk amount of data.
- To overcome this problem DMA direct memory access is used.
- Here the device controller transfers the entire block of data directly from local buffer to main memory without intervention of CPU.
- Only one interrupt is generated for entire block. While device controller performs the operations then CPU performs other work.

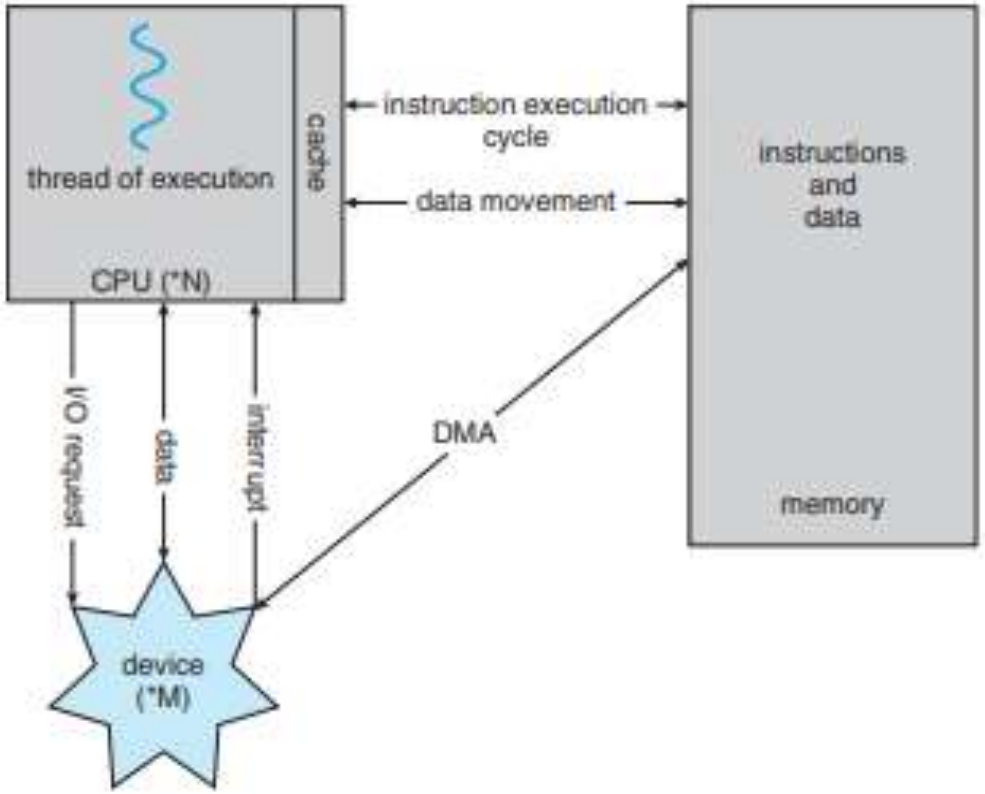


Figure 1.7 How a modern computer system works.

Storage structure

- The CPU can load instructions only from memory, so any programs must first be loaded into memory to run.
- General-purpose computers run most of their programs from rewritable memory, called main memory (also called random-access memory, or RAM).
- Main memory commonly is implemented in a semiconductor technology called dynamic random-access memory (DRAM).
- Boot strap programs are stored in ROM. It is read only memory once written not changeable.

- CPU consists of registers to store data. The load instruction moves data from main memory to CPU register.
- The execution cycle of von Neumann architecture first fetches instruction from main memory and store it in instruction register then it is decoded and may cause operands fetched from main memory and stored in internal registers. After execution completed the result send back to main memory.
- Cache memory which is between CPU and main memory which improves the execution speed of processor. It is fast memory.
- Secondary storage is an extension of main memory which stores the data permanently.

EX: Magnetic taps, hard disk, Magnetic disks..

Which are low access and cheaper devices.

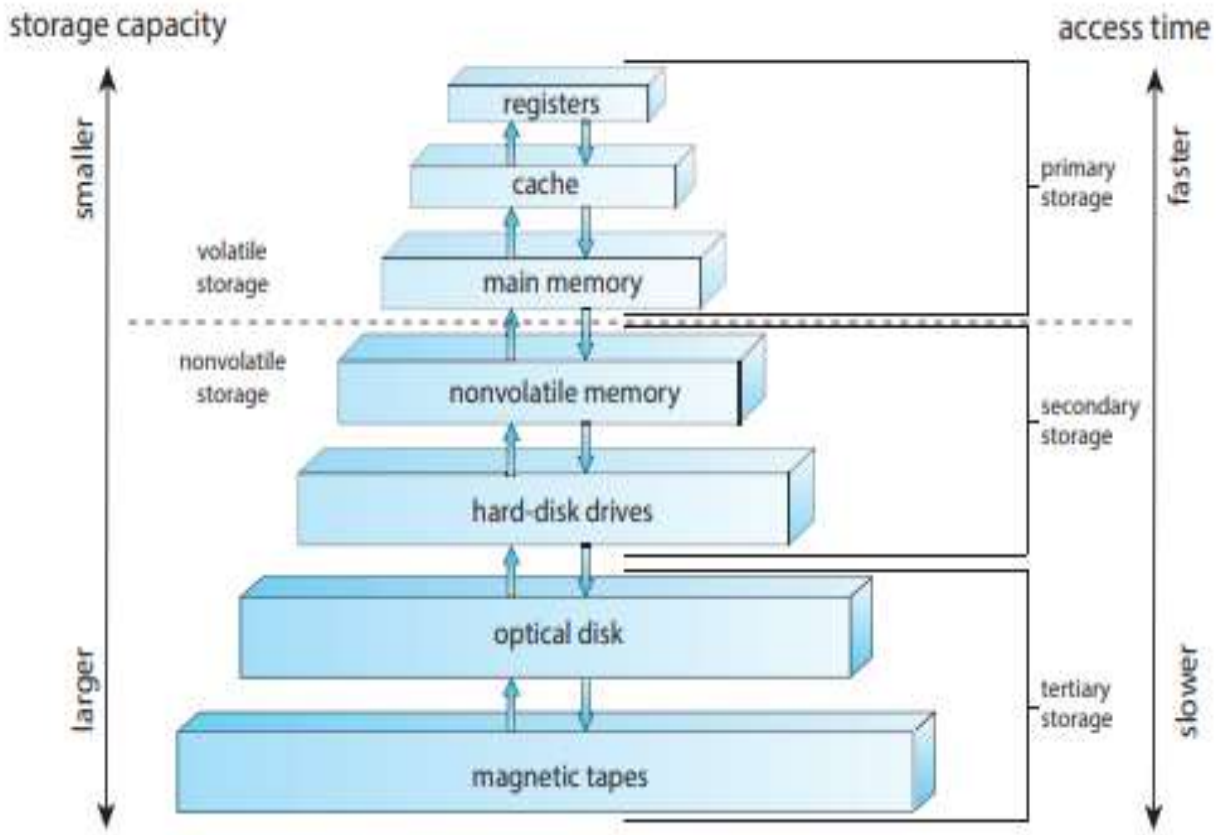


Figure 1.6 Storage-device hierarchy.

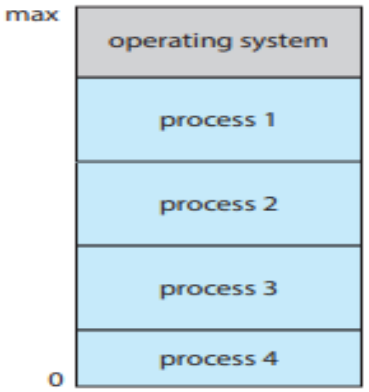
Operating-System Operations

- Multiprogramming and Multitasking
- Dual-Mode and Multimode Operation
- Timer

Multiprogramming and Multitasking

- One of the most important aspects of operating systems is the ability to run multiple programs, in general, keep either the CPU or the I/O devices busy at all times.
- Furthermore, users typically want to run more than one program at a time as well. Multiprogramming increases CPU utilization, as well as keeping users satisfied, by organizing programs so that the CPU always has one to execute. In a multiprogrammed system, a program in execution is termed a process
- The idea is as follows: The operating system

  keeps several processes in memory simultaneously

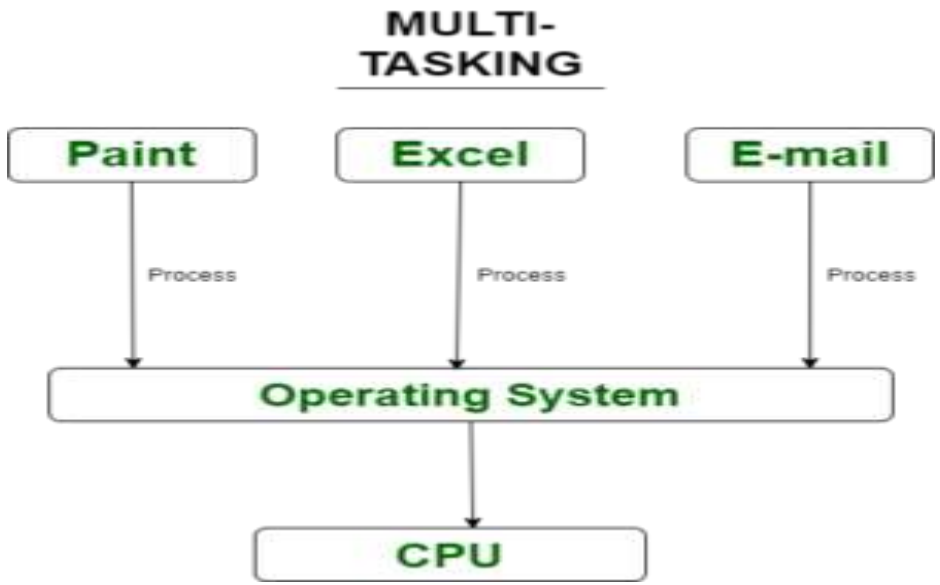


- The operating system picks and begins to execute one of these processes. Eventually, the process may have to wait for some task, such as an I/O operation, to complete.

- **In a non-multiprogrammed system, the CPU would sit idle.**

- In a multiprogrammed system, the operating system simply switches to, and executes, another process. When that process needs to wait, the CPU switches to another process, and so on. Eventually, the first process finishes waiting and gets the CPU back. As long as at least one process needs to execute, the CPU is never idle.

- **Multitasking** is a logical extension of multiprogramming. In multitasking systems, the CPU executes multiple processes by switching among them, but the switches occur frequently, providing the user with a fast response time. Consider that when a process executes, it typically executes for only a short time before it either finishes or needs to perform I/O



Imagine the Computer is a Toy Robot

- Your computer is like a robot with one brain (CPU).
- • It can do one thing at a time but switches tasks quickly.
- • We’ll learn two ways this robot works: Multiprogramming and Multitasking!

What is Multiprogramming?

- 3 kids want to play: A (racing), B (drawing), C (story).
- Kid A's game says 'loading' – robot goes to Kid B.
- If B is saving, it moves to C.
- Only switches when one is waiting – keeps robot busy.
- Goal: Don’t waste robot time!

What is Multitasking?

- All 3 kids are doing things at the same time.
- Robot gives 1 second each: A, then B, then C... repeat.
- Switches very fast – feels like all tasks run together.
- Even if no one is waiting, it switches.
- Goal: Make everyone feel the robot is fast and responsive.

Multiprogramming vs Multitasking

- Multiprogramming: Switch only when needed (e.g., waiting).
- Multitasking: Switch all the time, very fast.

- Multiprogramming: Feels like one job at a time.
- Multitasking: Feels like many jobs at once.

- Multiprogramming: Best for batch work.
- Multitasking: Best for interactive use (like you playing & typing).

- Having several processes in memory at the same time requires some form of **memory management.**

- if several processes are ready to run at the same time, the system must choose which process will run next. Making this decision is **CPU scheduling.**

- Multiprogramming and multitasking systems must also provide a file system .The file system resides on a secondary storage; hence, **storage management** must be provided.

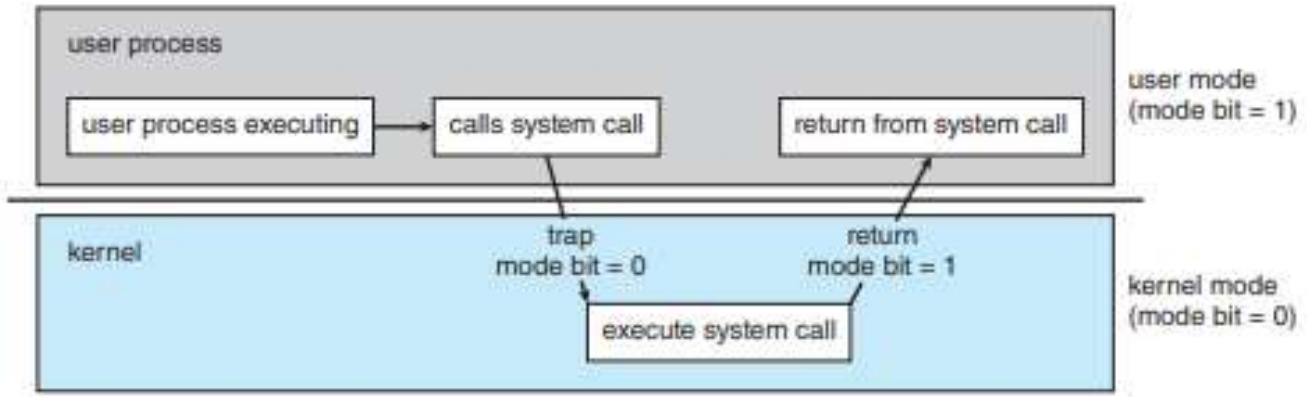
- To ensure orderly execution, the system must also provide mechanisms for **process synchronization and communication**

Dual-Mode and Multimode Operation

- **Reason::**Since the operating system and its users share the hardware and software resources of the computer system, a properly designed operating system must ensure that an incorrect (or malicious) program cannot cause other programs — or the operating system itself—to execute incorrectly.

- **In order to ensure the proper execution of the system, we must be able to distinguish between the execution of operating-system code and user-defined code.** The approach taken by most computer systems is to provide hardware support that allows differentiation among various modes of execution.

- At the very least, we need two separate modes of operation:
  - user mode
  - kernel mode (also called supervisor mode, system mode, or privileged mode).
- A bit, called the mode bit, is added to the hardware of the computer to indicate the current mode: kernel (0) or user (1).
- With the mode bit, we can distinguish between a task that is executed on behalf of the operating system and one that is executed on behalf of the user



- At system boot time, the hardware starts in kernel mode.

- The operating system is then loaded and starts user applications in user mode. Whenever a trap or interrupt occurs, the hardware switches from user mode to kernel mode (that is, changes the state of the mode bit to 0).

- Thus, whenever the operating system gains control of the computer, it is in kernel mode. The system always switches to user mode (by setting the mode bit to 1) before passing control to a user program.

1.Operating System (OS) Code Execution

- **Runs in: Kernel Mode**

- **Access:** Full access to **hardware resources**, **memory**, **CPU instructions**, and **I/O devices**.

- **Purpose:** Performs critical tasks like:
  - Managing memory
  - Handling system calls
  - Controlling hardware (disks, printers, network)
  - Scheduling processes
  - Managing file systems
- **Example:** Executing a system call like read() from disk.

2. User-Defined Code Execution

- **Runs in: User Mode**

- **Access: Limited access** – cannot directly interact with hardware or critical OS components.

- **Purpose:** Runs application-level programs like:
  - Word processors
  - Games
  - Web browsers
- **Relies on OS** for hardware-related operations via **system calls**.
- **Example:** A Python program printing "Hello, World!"

How They Interact:

- When a **user program** needs to perform a system-level task (e.g., open a file), it uses a **system call**.
- This causes a **mode switch** from **user mode to kernel mode**.
- The OS executes the required service, then **returns control** to the user program in **user mode**.

Example: A User Program Reading a File

Suppose you have a user-defined Python program that reads and prints the contents of a file called data.txt:

```
# User-defined code
with open("data.txt", "r") as file:
    content = file.read()
    print(content)
```

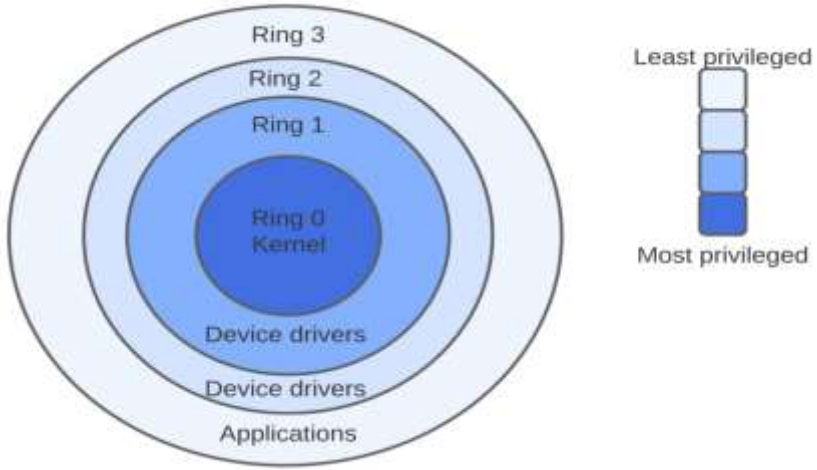
Step	What Happens	Who Executes It	Mode
1	The user runs the Python program.	User Program	User Mode
2	The program calls open("data.txt")	User Program	User Mode
3	Python triggers a system call to open the file.	Operating System	Kernel Mode
4	OS checks file permissions and locates the file.	Operating System	Kernel Mode
5	OS reads the file contents from disk to memory.	Operating System	Kernel Mode
6	System call returns control and file content.	Operating System	Switch to User Mode
7	Python program prints the file contents.	User Program	User Mode

Summary of Roles:

- **User-defined code** (Python script): Writes the logic to read and print a file, but **cannot access the disk directly**.

- **Operating System code**: Handles the actual file opening and reading by communicating with the **file system and disk**.

- The concept of modes can be extended beyond two modes.
- For example, Intel processors have four separate protection rings, where ring 0 is kernel mode and ring 3 is user mode.
- (Although rings 1 and 2 could be used for various operating-system services, in practice they are rarely used.
- ARMv8 systems have seven modes.
- CPUs that support virtualization (Section 18.1) frequently have a separate mode to indicate when the virtual machine manager (VMM) is in control of the system.
- In this mode, the VMM has more privileges than user processes but fewer than the kernel.



TIMER

- We must ensure that the operating system maintains control over the CPU.
- We cannot allow a user program to get stuck in an infinite loop or to fail to call system services and never return control to the operating system.
- To accomplish this goal, we can use a timer.
- A timer can be set to interrupt the computer after a specified period.
- The period may be fixed (for example, 1/60 second) or variable (for example, from 1 millisecond to 1 second).

- A variable timer is generally implemented by a fixed-rate clock and a counter.

- The operating system sets the counter. Every time the clock ticks, the counter is decremented. When the counter reaches 0, an interrupt occurs.

- Before turning over control to the user, the operating system ensures that the timer is set to interrupt.

- If the timer interrupts, control transfers automatically to the operating system, which may treat the interrupt as a fatal error or may give the program more time

Functions of operating system

- Process Management
- Memory Management
- Device Management(I/O)
- File Management
- Mass storage management
- Security
- Control over system performance
- Job accounting
- Error detecting aids
- Coordination between other software and users.

Process Management

- A program can do nothing unless its instructions are executed by a CPU.
- A program in execution, as mentioned, is a process.
- A program such as a compiler is a process, and a word-processing program being run by an individual user on a PC is a process. Similarly, a social media app on a mobile device is a process.
- A process needs certain resources—including CPU time,memory, files, and I/O devices—to accomplish its task.
- These resources are typically allocated to the process while it is running.

Process Management

In multiprogramming environment, the OS decides which process gets the processor when and for how much time. This function is called process scheduling. An Operating System does the following activities for processor management −

- Keeps tracks of processor and status of process. The program responsible for this task is known as traffic controller.
- Allocates the processor (CPU) to a process.
- De-allocates processor when a process is no longer required.
- Creating and deleting both user and system processes
- Suspending and resuming processes
- Providing mechanisms for process synchronization
- Providing mechanisms for process communication
- Providing mechanisms for deadlock handling

Memory Management

- Memory management refers to management of Primary Memory or Main Memory. Main memory is a large array of words or bytes where each word or byte has its own address.
- Main memory provides a fast storage that can be accessed directly by the CPU. For a program to be executed, it must in the main memory. An Operating System does the following activities for memory management −

- Keeps tracks of primary memory, i.e., what part of it are in use by whom, what part is not in use?
- In multiprogramming, the OS decides which process will get memory when and how much.
- Allocates the memory when a process requests it to do so.
- De-allocates the memory when a process no longer needs it or has been terminated.

Two types of memory allocations are there:

-continuous memory allocation: a)Memory fixed tasks b)Memory variable tasks

-noncontiguous memory allocation: a)paging b)segmentation

- Paging is a memory management scheme that **eliminates the need for contiguous allocation of physical memory**.

- When a process is executed, its pages are loaded into any available memory frames.

- A **page table** is used to map logical pages to physical frames.

- **Example:**
- If a process has 10 pages and RAM has free frames scattered, the OS can load these 10 pages into any 10 available frames, not necessarily contiguous.

- Segmentation is a memory management technique where memory is divided based on **logical divisions** like **functions, arrays, data, code**, etc.
- **Example:**

```
int globalVar;            // Global variable (Data segment)
void main() {
int localVar = 5;      // Local variable (Stack segment)
printf("Hello");   }    // Code (Text segment)
```

- A program might have:
  - Segment 0: Code
  - Segment 1: Data
  - Segment 2: Stack

  Each loaded into different parts of memory based on size and availability.

Device management(I/O)

- An Operating System manages device communication via their respective drivers. It does the following activities for device management −

- Keeps tracks of all devices. Program responsible for this task is known as the **I/O controller**.
- Decides which process gets the device when and for how much time.
- Allocates the device in the efficient way.
- De-allocates devices.

File Management

A file system is normally organized into directories for easy navigation and usage. These directories may contain files and other directions.

An Operating System does the following activities for file management −

- Keeps track of information, location, uses, status etc. The collective facilities are often known as **file system**.
- Creating and deleting files and directories
- Primitives to manipulate files and directories
- Mapping files onto secondary storage
- Backup files onto stable (non-volatile) storage media

Mass storage management

- Usually disks used to store data that does not fit in main memory or data that must be kept for a “long” period of time
- Proper management is of central importance
- Entire speed of computer operation attach on disk subsystem and its algorithms
- OS activities
  - Free-space management
  - Storage allocation
  - Disk scheduling
- Some storage need not be fast
  - Tertiary storage includes optical storage, magnetic tape
  - Still must be managed – by OS or applications
  - Varies between WORM (write-once, read-many-times) and RW (read-write)

Evolution of operating system

- Mid 1940-50 programmers directly interacted with computer hardware.
- Directly the programs are written in machine code and were loaded with Input device(card reader).
- The programmers examine the errors in register and main memory if the program proceeded to normal completion then output appeared on printer.

These early system suffers from two main problems

- Scheduling– fixed time slot is allocated for each user. If the user completes the task before the allocated time then CPU sits idle. If not finished in allocated time then forced to stop before resolving the problem.
- Set-up time

Most of the time spend for loading the compiler, source code into main memory and object code and linking both object code and common functions. Each of these steps includes mounting and dismounting tapes and decks. If error occurs repeat the same process from the beginning. So more amount of time will be spent for setting up the program to run.

This procedure is called serial processing.

Spooling

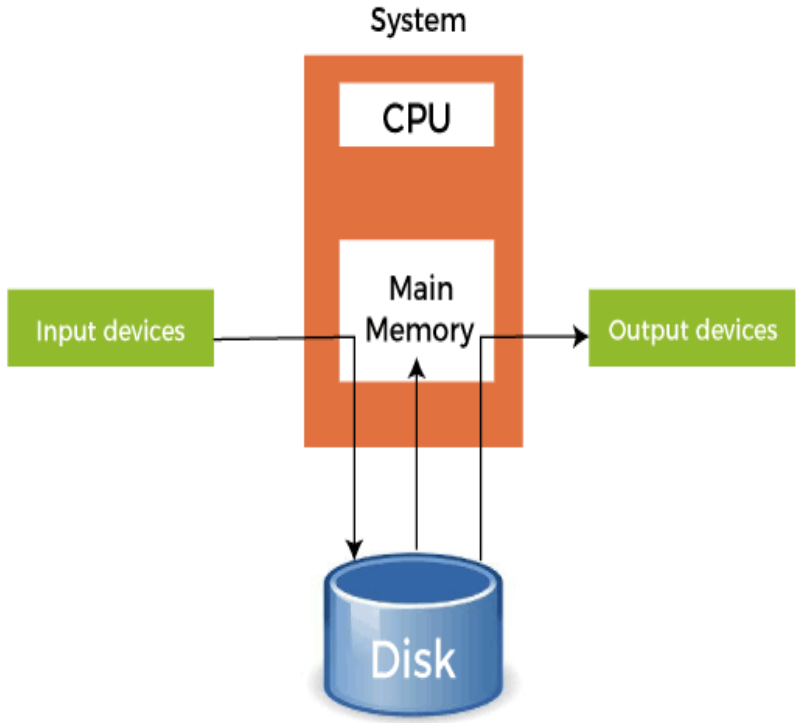
- Spooling stands for " **Simultaneous Peripheral Operations Online** ". So, in a Spooling, more than one I/O operations can be performed simultaneously i.e. at the time when the CPU is executing some process then more than one I/O operations can also de done at the same time.

- In this all the input devices data is collected and stored in disk. Then main memory fetches data from SM and CPU will execute some instructions on data.

- When the CPU generates some output, then that output is first stored in the main memory and the main memory transfers that output to the secondary memory and from the secondary memory, the output will be provided to some output devices.

- So I/O devices need not wait.

Example: In a printer spooling, there can be more than one documents that need to be printed. So, the documents can be stored into the spool and the printer can fetch that documents and print the document one by one.



- You are in your classroom, and your teacher has one printer for all the students. Everyone wants to print their drawings.
- But the printer can only print one drawing at a time. If all the kids shout, “Print mine! Print mine!”, the printer will get confused.
- So the teacher says:
- “Hey kids! Just drop your drawings in this special box. I will take them out one by one and print them.”
- That special box is like spooling!

In Simple Words:

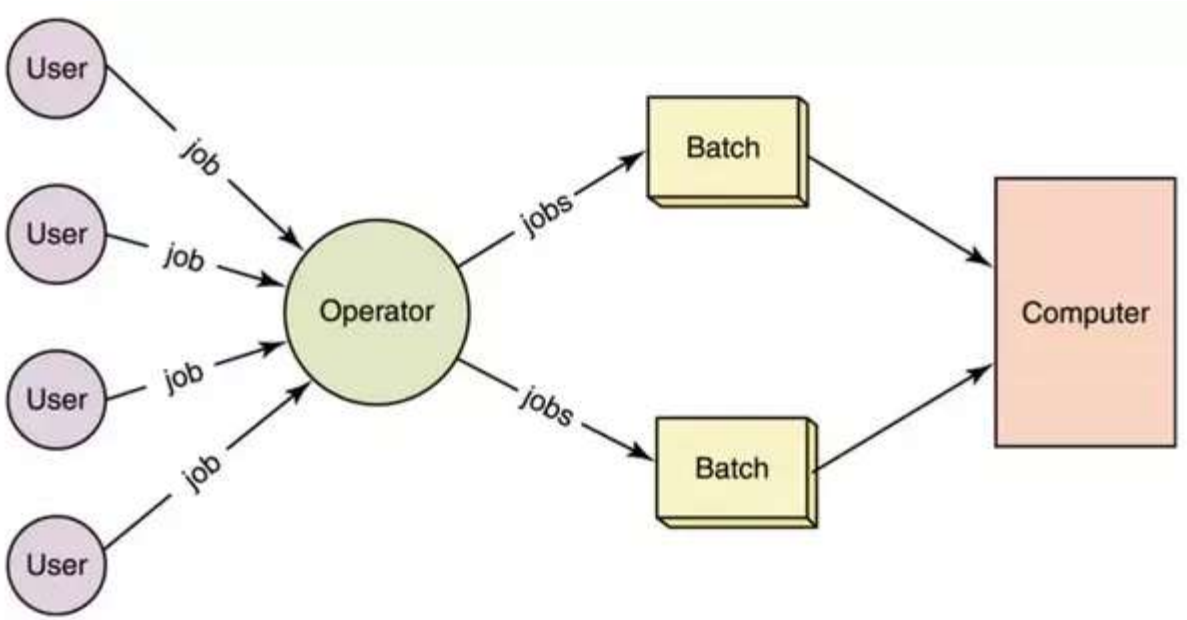
- Spooling means:
- Tasks (like print jobs) are kept in a queue (like the box in class).
- The computer sends each task to the printer one by one.
- While one job is printing, others wait their turn.

Simple batch operating system

- Used on IBM 1130.
- Punched cards as input devices.
- Programs written in the form of 0 and 1
- These cards are collected and each set of card is called a job.
- Set of jobs like job1, job2 ,..job20 are submitted as a batch.
- Only one job is kept in main memory.
- CPU executes only one job at a time after completion then it goes to next job soon.

- A batch operating system does not interact directly with the user. Instead, users prepare their jobs (programs + data) and submit them to the computer operator. The operator groups similar jobs into a batch and runs them all at once.

- Suppose 10 users submit print jobs. These are grouped together and executed one after the other. Users don't get immediate results—they have to wait until the whole batch is completed.

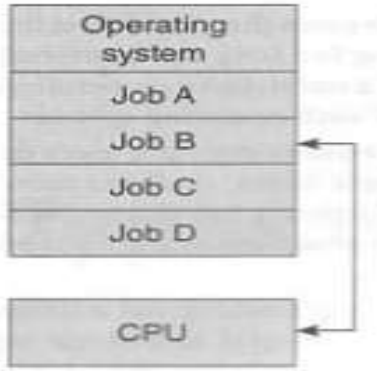


Disadvantages:

- Lack of interaction between the user and the job.
- CPU is often idle, because the speed of the mechanical I/O devices is slower than the CPU.
- Difficult to provide the desired priority.

Multi programming operating system

- In which memory contains more than one user program.
- In mono programming, CPU executing on program when I/O operation is encountered then CPU goes idle.
- The CPU utilization becomes poor
- To overcome this problem, MP OS were implemented.
- In which when one user program contains I/O operations then CPU switches to next user program.
- So CPU busy at all times.

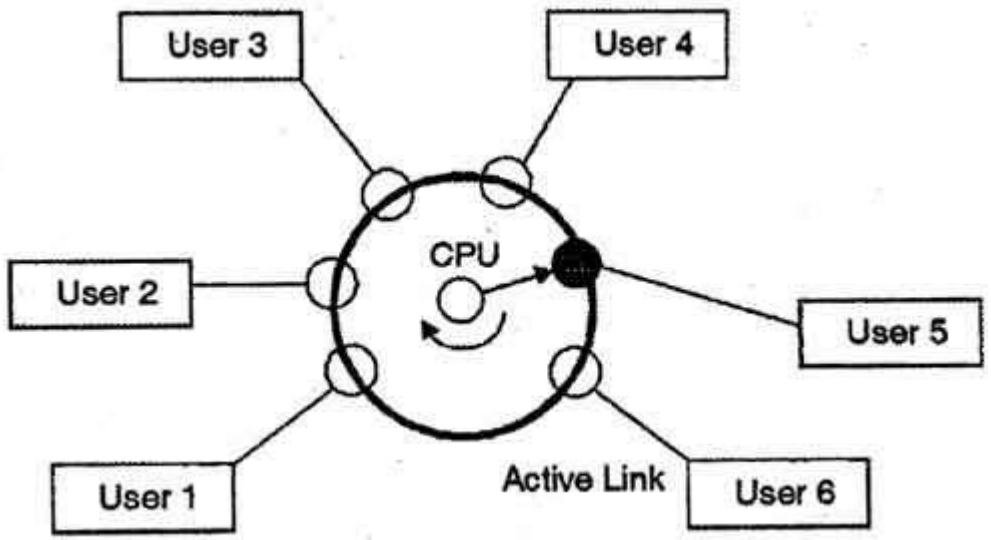


Advantages:

- CPU utilization is high
- Higher job throughput.
- Throughput=amount of time CPU utilized/Total time for executing the program.

Time sharing operating system

- It is logical extension of multiprogramming.
- which enables many people, located at various terminals, to use a particular computer system at the same time.
- Which minimizes the response time.
- Multiple jobs are executed by the CPU by switching between them, but the switches occur so frequently. Thus, the user can receive an immediate response.



- A time-sharing OS allows multiple users to use the system at the same time. It switches between tasks so quickly that each user thinks they are using the system exclusively.

- If 5 users are logged into a server, the OS gives each a time slice (e.g., 50 ms), and keeps switching between them rapidly.

- The operating system uses CPU scheduling and multiprogramming to provide each user with a small portion of a time.

Advantages of Timesharing operating systems are as follows −

- Provides the advantage of quick response.
- Avoids duplication of software.
- Reduces CPU idle time.

Disadvantages of Time-sharing operating systems are as follows −

- Problem of reliability
- Question of security and integrity of user programs and data.
- Problem of data communication.

Network operating System

- An operating system that provides the connectivity among a number of autonomous computers is called a network operating system.
- A typical configuration for a network operating system is a collection of personal computers along with a common printer, server and file server for archival storage, all tied together by a local network.
- Mainly there are two types of network operating systems
    - peer to peer
    - client/server

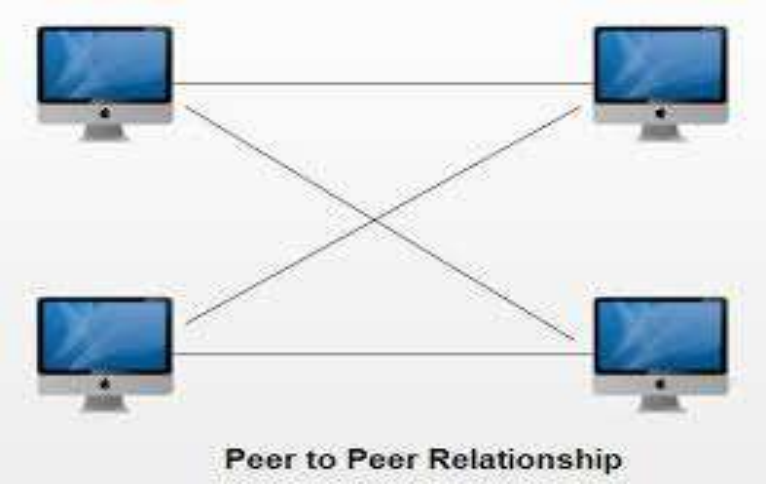
Peer to Peer

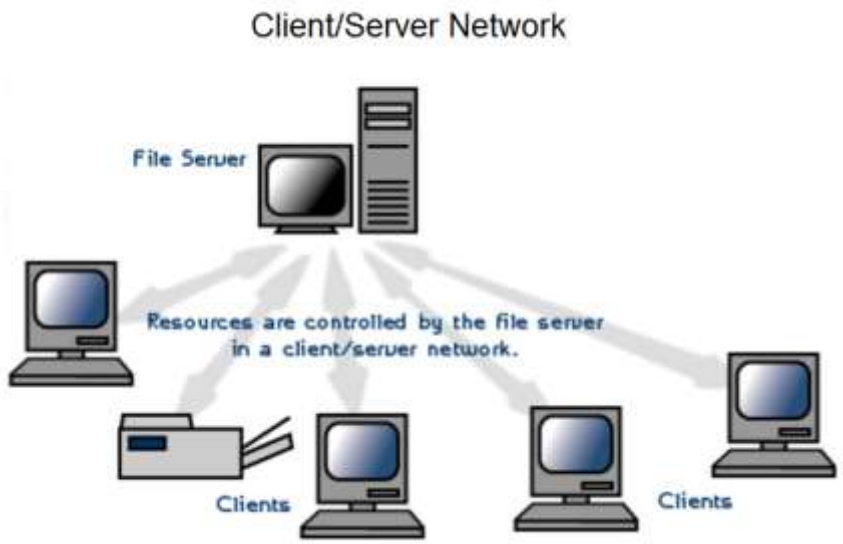
- It allow users to share resources and files located on their computers and to access shared resources found on other computers.
- In a peer-to-peer network, all computers are considered equal; they all have the same privileges to use the resources available on the network.
- Designed primarily for small to medium local area networks.

Ex: windows for workgroup.

Client/server

- Allow the network to centralize functions and applications in one or more dedicated file servers.
- The file servers become the heart of the system, providing access to resources and providing security.
- The workstations (clients) have access to the resources available on the file servers.
- The network operating system allows multiple users to simultaneously share the same resources irrespective of physical location.
- Example: Novell Netware and Windows 2000Server





- Examples of network operating systems include Microsoft Windows Server 2003, Microsoft Windows Server 2008, UNIX, Linux, Mac OS X, Novell NetWare, and BSD.

Advantages:

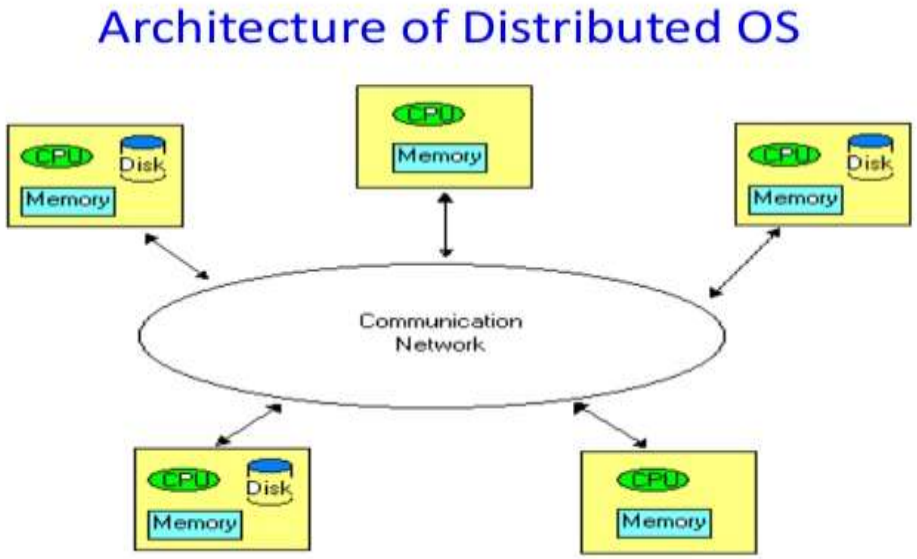
- Centralized servers are highly stable.
- Security is server managed.
- Upgrades to new technologies and hardware can be easily integrated into the system.
- Remote access to servers is possible from different locations and types of systems.

Dis-advantages:

- High cost of buying and running a server.
- Dependency on a central location for most operations.
- Regular maintenance and updates are required.

Distributed operating System

- It is extension of network operating system.
- Runs on multiple independent CPU'S.
- Uses multiple central processors to serve multiple real-time applications and multiple users.
- The processors communicate with one another through various communication lines (such as high-speed buses or telephone lines). These are referred as loosely coupled systems or distributed systems where each processor has its own local memory and processors communicate with one another through various communication lines, such as high speed buses or telephone lines.



- A distributed OS connects multiple computers (nodes) over a network and makes them function as a single system.

- If you submit a task, the OS may break it into parts and send them to different computers. After processing, it combines the results.

Advantages

- With resource sharing facility, a user at one site may be able to use the resources available at another.
- Speedup the exchange of data with one another via electronic mail.
- If one site fails in a distributed system, the remaining sites can potentially continue operating.
- Better service to the customers.
- Reduction of the load on the host computer.
- Reduction of delays in data processing.

Parallel(multi) processing operating systems

- In which multiprocessor systems, they have more than one processor sharing the computer bus, clock, memory and I/O devices.
- These are called tightly couples systems.
- Ex: UNIX for multimax computer.

Advantages:

Increases throughput.

Decreases cost

improves reliability

Operating system services

An operating system provides an environment for the execution of programs. Services of OS are

1. User Interface.
2. Program execution.
3. I/O operations.
4. File-system manipulation
5. Communications
6. Error detection
7. Resource allocation.
8. Logging
9. Protection and security

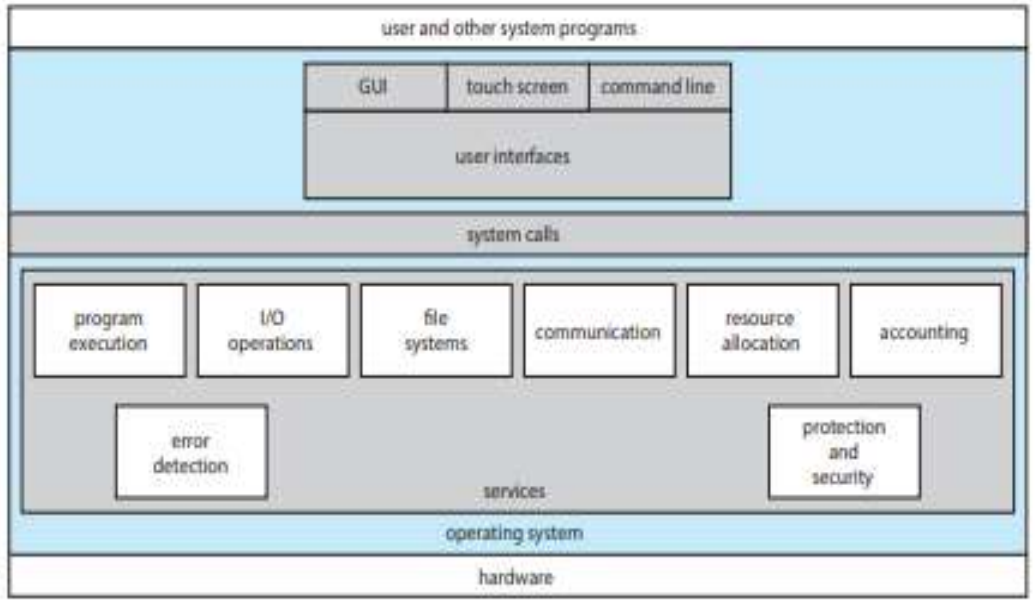


Figure 2.1 A view of operating system services.

1. **User interface :** Almost all operating systems have a **user interface** (UI). This interface can take several forms. One is a **command-line interface (CLI),** which uses text commands and a method for entering them. Another is a **batch interface,** in which commands and directives to control those commands are entered into files, and those files are executed. Most commonly a **graphical user interface (GUI)** is used. Here, the interface is a window system with a pointing device to direct I/O, choose from menus, and make selections and a keyboard to enter text.
2. **Program execution:** The system must be able to load a program into memory and to run that program. The program must be able to end its execution, either normally or abnormally (indicating error).
3. **I/O operations:** A running program may require I/O device. For specific devices, special functions may be desired (such as recording to a CD or DVD drive). For efficiency and protection, users usually cannot control I/O devices directly. Therefore, the operating system must provide a means to do I/O.
4. **File-system manipulation:** Operating System is used to control operations on files like creating, opening, reading, and writing.
5. **Communications:** There are many cases in which one process needs to exchange information with another process. Such communication may occur between processes that are executing on the same computer or between processes that are executing on different computer systems tied together by a computer network. Communications may be implemented via *shared memory* or through *message passing*, in which packets of information are moved between processes by the operating system.

6. Error detection. The operating system needs to be constantly aware of possible errors. Errors may occur in the CPU and memory, in I/O devices, and in the user program. For each type of error, the operating system should take the appropriate action to ensure correct and consistent computing.

7. Resource allocation. When there are multiple users or multiple jobs running at the same time, resources must be allocated to each of them.

8. Accounting: Accounting keep track of which users use how much and what kinds of computer resources. This record keeping may be used for accounting (so that users can be billed) or simply for accumulating usage statistics.

9. Protection and security. The owners of information stored in a multiuser or networked computer system may want to control use of that information. When several separate processes execute concurrently, it should not be possible for one process to interfere with the others or with the operating system itself. Protection involves ensuring that all access to system resources is controlled. Security of the system from outsiders is also important. Such security starts with requiring each user to authenticate himself or herself to the system, usually by means of a password, to gain access to system resources.

User and operating systems interface

The command interpretation module (also called command interpreter) provides a set of commands that the users can execute.

These commands are called user interface.

When user executes a system call, the command interpreter interprets the instructions and allocates the resources to handle the user's request.

There are three types of user interfaces

- Command line interface
- Graphical user interface
- Touch screen interface

Command Line Interface(CLI):

- CLI is also called as command line user interface or console user interface or character user interface.
- User interact with the system by typing command at command prompt.

Graphical user Interface(GUI):

- GUI provides graphics components to interact with the computer.
- Need not type or remember any command.
- User select commands and objects by selecting with mouse.
- User interface commands are displayed in menus.
- It facilitates that the users can work in multiple windows each window represents different applications.
- GUI's are developed for both desktop, laptop and other mobile computing devices.

Touch screen interface

- In this users interact with the device by hand gestures on the screen.
- Although earlier smartphones included a physical keyboard, most smartphones and tablets now simulate a keyboard on the touch screen.
- Both the iPad and the iPhone use the Springboard touch-screen interface.

Example systems

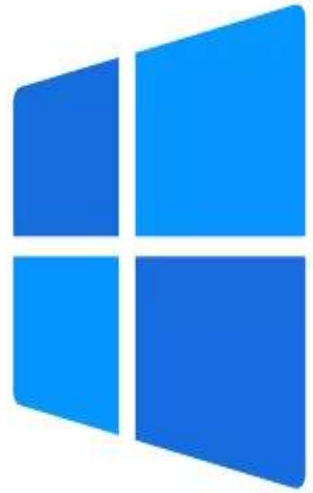
- MS Windows
- MS DOS
- Ubuntu OS
- Mac OS
- Apple IOS
- Linux OS
- UNIX OS
- Android OS
- Chrome OS
- Fedora OS

MS-Windows

- Windows is a personal computer operating system. It was launched by Microsoft on November 20, 1985.
- Both GUI and CLI Interface.
- Supports multiprogramming.

Different windows versions are

- Windows NT
- Windows XP
- Windows Vista
- Windows 7
- Windows 8 and 8.1
- Windows 10
- Windows 11



MS DOS

- It is an operating system made by Microsoft that is the predecessor to Windows. MS-DOS was officially launched on August 12, 1981, by Microsoft.
- Provides CLI environment only.
- Supports single programming operating system.

Ubuntu OS

- It is an open-source Linux-based operating system. It can be installed on desktops, servers, and smartphones.
- Provides both CLI and GUI.
- Supports multi-programming system.

Mac OS

- It is a desktop operating system created by Apple Inc.
- that has been designed to work on Macintosh computers. The original version of Mac OS was introduced in 1984. Mac OS is also known as the Macintosh operating system.
- Mac OS provides a number of different features, such as a graphical user interface, pre-emptive multitasking, and memory protection.

Apple iOS

- Apple IOS is a mobile operating system that was created in 2007 and released in 2008. It was the successor of Apple's other operating system, known as OS X.
- The IOS has been developed by Apple Inc., which is headquartered in Cupertino, California.
- IOS is designed for Apple devices, such as the iPhone iPad Pro, and iPod Touch.

Linux OS

- Linux is a free and open-source operating system.
- This operating system was created by Linus Torvalds and h community of many developers and this Linux operating system was launched on September 17 1991.
- Linux operating system is specially designed for personal computers and is also a popular operating system after Windows and Mac OS.

UNIX

- UNIX is a computer operating system originally developed in 1969 and was built with the intention of being an open and free operating system.
- This operating system was created by many developers, whose names are Ken Thompson, Dennis Ritchie, Brian Kernighan, Douglas McIlroy, and Joe Ossanna. Unix Operating System was developed at Bell Labs Research Center.
- It is a multi-user, multitasking, and multithreaded computer system that is capable of running different applications at the same time.
- Unix also has many different versions and this operating system is based on the graphical user interface.
- This operating system is widely used in many modern computers such as - workstation computers, servers, mobile phones, etc.
- There are many components of Unix OS like - Kernel, the Shell, the programs, etc.

Android OS

- It is the most popular mobile-based operating system in the world.
- The Android operating system is developed by a consortium of developers known as the Open Handset Alliance and commercially sponsored by Google.
- Android OS was launched on September 23, 2008. Initially, the Android operating system was designed only for smartphones.
- OS has been used in many devices such as smartphones, tablets PC, smartwatches, vehicles, digital cameras, TV, etc. It comes with a touchscreen device which attracts people even more.

Chrome OS

- Chrome OS is a Google operating system that focuses on the Google Chrome web browser user interface.
- Chrome OS is a popular Linux-based operating system which is developed and designed by Google and initially released on June 15, 2011.
- Chrome OS has been developed exclusively for Chromebook users. Initially, Chrome OS was developed for the purpose of using web applications and there are many versions of Chrome OS.

Fedora OS

- Fedora is a Linux-based operating system that is distributed free of charge.
- Fedora is a personal computer-based operating system designed to perform general-proposed tasks.
- Fedora OS was developed under the Fedora Project and sponsored by Red Hat, an IBM subsidiary that was finally released in 2003.